



山东大学  
SHANDONG UNIVERSITY

# 计算机视觉实验报告

基于计算机视觉的工业装配件缺失检测

学院 低空科学与工程学院

班级 23 计算机科学与技术 01

学号 202300800153

姓名 李龙祺

2026 年 4 月 27 日

# 目录

|          |                                   |          |
|----------|-----------------------------------|----------|
| <b>1</b> | <b>任务背景与要求</b>                    | <b>1</b> |
| <b>2</b> | <b>总体方案</b>                       | <b>1</b> |
| <b>3</b> | <b>预处理与分件</b>                     | <b>1</b> |
| 3.1      | 稳定工件检测 . . . . .                  | 1        |
| 3.2      | Task A 动态定位 . . . . .             | 2        |
| 3.3      | Task B 多帽位 ROI . . . . .          | 2        |
| <b>4</b> | <b>核心算法</b>                       | <b>2</b> |
| 4.1      | DINOv2 特征提取 . . . . .             | 2        |
| 4.2      | PatchCore 记忆库 . . . . .           | 2        |
| 4.3      | Task B 独立 ROI PatchCore . . . . . | 2        |
| <b>5</b> | <b>实验结果</b>                       | <b>3</b> |
| 5.1      | 演示视频 . . . . .                    | 3        |
| 5.2      | Task A 结果 . . . . .               | 3        |
| 5.3      | Task B 结果 . . . . .               | 3        |
| <b>6</b> | <b>速度与工程实现</b>                    | <b>3</b> |
| <b>7</b> | <b>困难情况与改进</b>                    | <b>4</b> |
| <b>8</b> | <b>结论</b>                         | <b>4</b> |

## 1 任务背景与要求

本项目完成两段工业流水线视频中的装配件缺失检测。视频前段只包含正常装配样本，测试段同时包含正常件与异常件。系统需要在不使用异常样本训练的条件下，对每个稳定到位的工件输出 OK 或 NG，并在演示视频中显示 ROI、检测分数与状态。

作业指南要求包含三部分产物：预处理、训练和推理代码；能够展示缺盖案例的短演示视频；以及说明方法、结果和改进思路的 PDF 报告。本项目的代码入口为 `tools/run_demo.py`，核心模块位于 `src/`，最终演示视频输出到 `results/`。

## 2 总体方案

系统采用无监督异常检测路线。训练阶段只从正常装配片段中提取稳定工件的 ROI patch，使用预训练 DINOv2 ViT-S/14 提取 patch token 特征，并用 PatchCore 记忆库进行最近邻异常评分。测试阶段按工件分组，对稳定区间采样评分，并聚合为单件 OK/NG 结果。

整体流程如下：

1. 跳过视频前 60 秒抖动片段。
2. 根据固定 ROI 或动态定位得到检测区域。
3. 通过运动、模糊、前景与存在性门控，只在工件稳定到位时检测。
4. 对训练段正常 ROI 提取 DINOv2 patch 特征，建立 PatchCore memory bank。
5. 对测试段每件工件计算异常分数，超过阈值判定为 NG。
6. 使用仪表盘视频显示当前状态、分数、ROI 框与 OK/NG 结果。

## 3 预处理与分件

### 3.1 稳定工件检测

流水线视频中存在空场景、进入和离开过程、运动模糊以及工件只露出一部分的情况。若逐帧检测，会产生大量误报。因此本项目实现了按件检测：当工件进入 ROI 后，UnitTracker 根据帧差、前景比例、模糊度和最小稳定帧数建立一个工件单元；只有单元稳定时才送入检测器。

对于无工件或半进入状态，系统只显示 WAIT 或 MOVING，不输出 OK/NG，从而符合实际质检中“到位后判定”的要求。

### 3.2 Task A 动态定位

Task A 中圆形端面位置会有轻微漂移。项目在固定参考 ROI 的基础上加入动态定位模块 `src/locator.py`，在每帧中定位端面区域并裁剪对齐 patch。该处理降低了位置漂移对 PatchCore 分数的影响。

Task A 当前使用动态 ROI、DINOv2 特征和 PatchCore 判定，并额外通过端面颜色比例辅助识别盖子缺失后的黄色内衬区域。

### 3.3 Task B 多帽位 ROI

Task B 的一个工件包含 4 个关键帽位。项目为 4 个帽位分别设置 ROI，并在视频旋转 180 度后同步旋转 ROI。4 个帽位角色为：蓝、白、白、蓝。系统要求至少 3 个帽位可见且稳定时才进入检测，避免半进入或半离开工件被误判。

## 4 核心算法

### 4.1 DINOv2 特征提取

项目使用 `timm` 加载 `vit_small_patch14_dinov2`。每个 ROI 被缩放到统一输入尺寸，送入模型后取 patch token 作为局部特征。与手工颜色阈值相比，DINOv2 特征能同时表达边缘、结构、纹理和部件形态；与有监督检测器相比，它不需要异常标注。

### 4.2 PatchCore 记忆库

训练段中的正常 patch 特征构成候选特征集合。为降低存储和 kNN 搜索开销，系统使用 `coreset` 子采样压缩 memory bank。测试时，对每个 patch 特征计算其到正常 memory bank 最近邻的距离，并按 `mean_top5` 聚合为 ROI 异常分数。

阈值由训练段正常分数自适应估计：

$$\theta = \mu_{train} + k \cdot \sigma_{train} \quad (1)$$

其中  $k$  由配置文件中的 `threshold_k` 控制。

### 4.3 Task B 独立 ROI PatchCore

Task B 若把四个帽位混在同一个 memory bank 中，会削弱每个帽位的局部差异。因此最终方案对四个 ROI 分别训练 PatchCore，每个帽位拥有独立阈值。单件检测时，只要任一 ROI 的聚合分数超过对应阈值，就判定该工件为 NG。

此外，Task B 针对蓝色和白色帽位分别计算颜色存在性比例。颜色存在性不作为主判定，而作为 PatchCore 的兜底信号：只有持续、明显低于训练正常分布时才触发

NG。最终配置中 `taskB_presence_ng_vote_ratio=0.90`，避免轻微反光或白色区域波动造成误报。

## 5 实验结果

### 5.1 演示视频

最终演示视频位于：

- `results/taskA_demo_result.mp4`
- `results/taskB_demo_result.mp4`

演示视频包含工件状态、OK/NG 结果、异常分数、阈值和 ROI 框。空场景与运动过程显示为等待或移动状态，不作为质检结果计数。

### 5.2 Task A 结果

Task A 使用动态定位后的单 ROI PatchCore。训练段仅使用正常装配样本，测试时按件聚合。经过动态定位后，端面漂移导致的误报显著减少；盖子缺失时，端面纹理和颜色变化会使 PatchCore 分数升高，并被判定为 NG。

### 5.3 Task B 结果

Task B 最终使用 1040s--1075s 附近片段作为短演示窗口，该窗口包含两个明显异常工件。50 秒源视频回归时，系统输出 10 件，其中 8 件 OK、2 件 NG；两个 NG 分别对应 1047 秒和 1072 秒附近的异常工件。生成 15 秒提交演示时，使用同一异常区间的 2 倍速短片。

| 片段                      | 工件数 | OK | NG |
|-------------------------|-----|----|----|
| Task B 1040s-1090s 回归片段 | 10  | 8  | 2  |
| Task B 250s-300s 回归片段   | 8   | 6  | 2  |
| Task B 930s-980s 压力测试片段 | 11  | 8  | 3  |

表 1: Task B 回归测试结果

## 6 速度与工程实现

系统按 `step_test=3` 抽帧检测，并对一个工件内部的稳定帧进行采样聚合，避免逐帧重复计算。PatchCore 使用 `coreset` 压缩 memory bank，减少 kNN 搜索成本。演示视

频以 2 倍速输出，当前配置下 Task B 30 秒源视频生成约 15 秒演示视频，满足短视频展示要求。

主要代码结构如下：

- `src/preprocessing.py`: 视频读取、旋转、ROI 裁剪、训练样本提取。
- `src/tracker.py`: 工件稳定状态机与按件分组。
- `src/locator.py`: Task A 动态端面定位。
- `src/feature_extractor.py`: DINOv2 特征提取。
- `src/memory_bank.py`: PatchCore memory bank、coreset 和 kNN 评分。
- `src/detection.py`: 单 ROI 与多 ROI 异常检测器。
- `src/visualization.py`: 演示视频仪表盘与 ROI 可视化。
- `tools/run_demo.py`: 训练、测试、可视化视频生成入口。

## 7 困难情况与改进

本任务中最难处理的是运动过程、反光、半进入工件和多帽位差异。Task A 的主要问题是端面位置漂移，因此加入动态定位。Task B 的主要问题是四个帽位外观不同，因此采用 per-ROI PatchCore，而不是把所有帽位混成一个模型。

后续可继续改进的方向包括：

- 将 DINOv2 特征缓存到磁盘，减少重复生成演示视频时的训练时间。
- 使用 FAISS 或 ONNX/TensorRT 加速最近邻搜索和特征推理。
- 为 Task B 增加少量可解释的结构模板，仅用于展示异常位置，不替代 PatchCore 主判定。
- 引入自动评估脚本，对已知异常时间点计算召回率和误报率。

## 8 结论

项目完成了从视频预处理、ROI 定位、无监督模型训练、按件推理到结果可视化的完整流程。Task A 通过动态定位提升了稳定性，Task B 通过多 ROI PatchCore 解决了四个帽位差异明显的问题。最终产物满足作业指南中对代码、演示视频和报告的要求，并保持训练阶段只使用正常样本。